

Universidad de los Andes

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Métodos Estadísticos para la Gestión

Proyecto 2 — Informe 1

Integrantes

Agustín Álvarez

Arantxa Toledo

Samuel Vallete

Profesor

Jorge Gomez Miranda

Informe 1 — Arquitectura del juego, contador de ciclos y temporización de 3 segundos

Introducción

Este informe documenta la arquitectura del sistema desarrollado para la entrega del segundo proyecto del ramo de Arquitectura de Computadores. Consiste en un juego de reflejos implementado sobre Pochoco SoC, e instanciado en una FPGA Lattice iCE40 HX1K (Go Board). El juego mismo se desarrolló en assembly para el Espino Core, y se tradujo a código máquina mediante un ensamblador propio.

El objetivo de este documento es describir el hardware y su arquitectura, detallando las modificaciones que se realizaron sobre el RTL base del repositorio para poder soportar el juego, y la forma en la la que el contador implementado se utiliza para resolver una temporización de tres segundos y la medición de tiempos de respuesta.

Este informe cubre tres aspectos principales: La arquitectura general del sistema y su integración en el Espino Core, memoria, y periféricos, las modificaciones hechas al RTL proveniente del repositorio base, y la justificación numérica de la conversión entre ciclos de reloj y las unidades de tiempo que se presentan al usuario.

Arquitectura general del sistema

El sistema se organiza en torno al Pochoco SoC, que se instancia dentro del módulo de nivel superior game_top, el cual se sintetiza y carga en la FPGA. game_top expone hacia el exterior los pines físicos de la placa, como el reloj, los LEDs, y los switches, y los conecta directamente a los puertos del SoC. En otras palabras, cablea el SoC al exterior.

Dentro de pochoco_soc conviven cuatro bloques principales:

- **Espino Core:** el procesador RV32E, el cual ejecuta el programa game.s. Emite una interfaz de instrucciones y una interfaz de datos para acceder a memoria y periféricos.
- **RAM unificada:** es la memoria compartida de instrucciones y datos, que se maneja en pochoco_ram. Se inicializa en el tiempo de síntesis del contenido a través del parámetro de MemFile. El puerto A atiende al fetch de instrucciones del core; el puerto B atiende al fetch de datos cuando la dirección no está en la zona de memoria de los periféricos.
- **Módulo de periféricos:** Se encuentra en pochoco_periph. Se encarga de agrupar todos los periféricos que usa el juego, es decir, los displays de 7 segmentos, LEDs, botones, y el contador de ciclos.

- **Interfaz SPI slave:** Se encuentra en pochoco_spi_slave. Expone un canal de comunicación con un SPI maestro externo, independiente de la lógica del juego.

El enrutamiento entre los bloques se resuelve por decodificación de direcciones: el bit más significativo de la dirección de datos distingue entre el espacio RAM y el de E/S mapeado en memoria. En el segundo, hay otro bit que separa los periféricos de la placa con el de la interfaz SPI. Un registro de un ciclo (rd_is_per, rd_is_spi) recuerda cada vez que se lee cual de los 3 bloques debería entregar el dato de vuelta al core cuando lo pide.

Todo el sistema es síncrono a un reloj de 25 MHz, y usa un reset de encendido generado internamente por un contador de 4 bits que mantiene el sistema en reset durante los primeros 16 ciclos tras la energización, asegurando que se conocen todos los registros cuando se arranca la ejecución.

Todo el sistema es síncrono a un único reloj (i_Clk, 25 MHz) y usa un reset de encendido (power-on reset) generado internamente por un contador de 4 bits que mantiene el sistema en reset durante los primeros 16 ciclos tras la energización, asegurando que todos los registros arrancan en un estado conocido.

Modificaciones al RTL

El repositorio base no incluye ningún mecanismo para medir el tiempo respecto a la cantidad de ciclos de reloj transcurridos. Para implementar el juego, se debió agregar esa capacidad, y la modificación se centró en el archivo pochoco_periph.v. En el código, ambos bloques añadidos están comentados y marcados explícitamente.

Contador de ciclos de 32 bits

Se agregó un registro cycle_q de 32 bits que se incrementa en 1 en cada flanco de subida de clk_i:

```
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cycle_q <= 32'b0;
  else         cycle_q <= cycle_q + 32'd1;
end
```

Este registro no es editable mediante software. En el bloque de escritura de periféricos, el case qu decodifica el offset de escritura no incluye ninguna entrada que sea capaz de llevar a ese registro, por lo que no se puede acceder para edición. Esto es intencional, ya que su único propósito es ser una base de tiempo de exclusivamente lectura.

Mapeo en memoria y lectura

El contador se expone al programa en el offset 0x0C (off = 3) dentro del espacio de periféricos, agregando una nueva entrada al case de lectura ya existente:

```
6'd3: rdata_o <= cycle_q;  // Cycle counter (offset 0x0C)
```

Desde game.s, el programa lo lee simplemente con una instrucción lw sobre la base de periféricos más el offset 12 (constante CICLOS en el código), como la lectura de cualquier

otro periférico.

Costo en área y comportamiento de desborde

El costo de esta modificación es acotado y queda documentado en el propio RTL: 32 flip-flops para almacenar el valor del contador más un sumador de 32 bits para incrementarlo en cada ciclo, sin lógica combinacional adicional relevante. Al ser un contador libre de 32 bits sin reset por software, vuelve a 0 cada 2^{32} ciclos — a 25 MHz, esto ocurre aproximadamente cada 171,8 segundos. Esto no afecta la lógica del juego porque todas las mediciones de tiempo se calculan como diferencias (t1 − t0) usando resta en aritmética módulo 2^{32}, la cual sigue dando el resultado correcto aun cuando el contador se haya desbordado entre ambas lecturas, siempre que el intervalo medido sea menor a 2^{32} ciclos (una ronda del juego dura, como máximo, unos pocos segundos).

Por qué este diseño y no una alternativa

Se optó por un contador de ciclos crudo, en lugar de, por ejemplo, un temporizador con interrupciones o un divisor de frecuencia dedicado, porque es la solución más simple y de menor costo en LUTs/flip-flops para una FPGA de recursos limitados como la iCE40 HX1K, y porque le da al software control total sobre qué intervalos medir (3 segundos de espera, décimas de segundo, tiempo de reacción), sin necesitar lógica adicional en hardware para cada caso de uso.

Frecuencia de reloj del sistema

Como ya fue mencionado, la Go Board no incluye un generador de reloj interno en el RTL del proyecto base. Por lo tanto, se usa Clk directamente como el reloj del sistema, sin multiplicar ni dividir la frecuencia.

El proyecto asume un oscilador de 25MHz como fuente de i_Clk. Todo el cálculo de temporización se basa en ese valor.

Trabajar directamente con el reloj de la placa, sin un divisor de frecuencia adicional, simplifica el diseño: evita agregar lógica de generación de reloj que consumiría recursos adicionales de la FPGA, un dispositivo de gama de entrada con recursos lógicos limitados. A cambio, toda la conversión de “ciclos de reloj” a “unidades de tiempo humanas” (décimas de segundo, segundos) se resuelve en software, usando el contador de ciclos — una decisión consistente con la filosofía general del proyecto de mantener el hardware simple y trasladar la complejidad de temporización al programa en assembly.

Conversión de ciclos a décimas de segundo

Con la frecuencia de reloj asumida de 25 MHz, cada ciclo de reloj dura:

T_ciclo = 1 / 25.000.000 Hz = 0,04 µs = 40 ns

A partir de esto, el número de ciclos necesarios para cubrir una décima de segundo (0,1 s)

se obtiene directamente:

ciclos_por_décima = 0,1 s / T_ciclo = 0,1 s × 25.000.000 Hz = 2.500.000 ciclos

Este valor corresponde exactamente a la constante TENTH definida en game.s:

```
.equ TENTH, 2500000   # ciclos por décima de segundo
```

De la misma forma, el número de ciclos necesarios para cubrir los 3 segundos de espera inicial se calcula como:

ciclos_3s = 3 s × 25.000.000 Hz = 75.000.000 ciclos

Valor que corresponde a la constante TRES_S del programa:

```
.equ TRES_S, 75000000   # ciclos de la fase de espera (3 s)
```

Unidad de tiempo elegida: décimas de segundo

El enunciado exige mostrar el tiempo de reacción del jugador en los displays de 7 segmentos, en formato decimal. Dado que los displays del proyecto son de dos dígitos y no hay forma de mostrar una coma decimal, se eligió la décima de segundo como unidad base de medición y despliegue: un valor como “23” en los displays representa 2,3 segundos de tiempo de reacción. Esta unidad es lo suficientemente fina para diferenciar reflejos humanos típicos (del orden de 150–400 ms) sin exceder el rango de dos dígitos decimales (0–99 décimas, es decir, hasta 9,9 s), y permite implementar la conversión de ciclos a unidad mostrada con una simple división entera por TENTH, hecha por resta repetida dado que la ALU no soporta shifts ni multiplicación/división nativas.

Mecanismo de temporización de 3 segundos

Al inicio de cada ronda, el programa debe mantener los 4 LEDs encendidos durante al menos 3 segundos antes de mostrar el LED objetivo. Esto se implementa en game.s con un bucle de espera activa (polling) que compara directamente contra el contador de ciclos de hardware:

```
addi x5, x0, 15
sw   x5, LEDS(x1)
lw   x9, CICLOS(x1)      # t0 de la fase de espera
espera_3s:
   lw   x10, CICLOS(x1)
   sub  x5, x10, x9         # elapsed = t1 - t0 (resta módulo 2^32)
   bltu x5, x13, espera_3s  # aún no pasan los 3 s
```

Paso a paso, la lógica es:

- Encender los 4 LEDs: se escribe el valor 15 (0b1111) en el offset de LEDs, encendiéndolos todos simultáneamente.
- Tomar la marca de tiempo inicial t0: se lee el contador de ciclos una vez y se guarda en x9, justo antes de entrar al bucle de espera.

- Bucle de espera (polling): en cada iteración se vuelve a leer el contador de ciclos (t1, en x10) y se calcula elapsed = t1 − t0 mediante una resta de 32 bits sin signo.
- Condición de salida: mientras elapsed sea menor que TRES_S (75.000.000 ciclos, calculado en la sección anterior), el programa vuelve a saltar al inicio del bucle (bltu x5, x13, espera_3s); en el momento en que elapsed ≥ TRES_S, el bucle termina y el programa continúa con la selección del LED objetivo.

Por qué la resta módulo 2^{32} es válida aunque el contador se desborde

El contador de ciclos (cycle_q) es de 32 bits y se desborda (vuelve a 0) cada 2^{32} ciclos (o cada ~171,8 segundos) a 25 MHz. Esto podría parecer un problema si, por ejemplo, t0 se toma justo antes del desborde y t1 se toma justo después. Sin embargo, la resta en aritmética de 32 bits sin signo sigue entregando el resultado correcto por la forma en que funciona la aritmética modular: si t0 = 0xFFFFFFF0 y el contador se desborda y llega a t1 = 0x00000005, la resta t1 − t0 calculada módulo 2^{32} da exactamente 21 (el número real de ciclos transcurridos), no un número negativo ni un valor corrupto. Esta propiedad es válida siempre que el intervalo real medido sea menor a 2^{32} ciclos, condición que se cumple ampliamente en este proyecto: la espera más larga del programa es de apenas 75.000.000 ciclos (3 segundos), muy por debajo del rango de desborde del contador. Por eso el programa no necesita ninguna lógica especial para manejar el wrap-around: la resta sin signo ya lo resuelve por su propia lógica.

Este mismo patrón de tomar t0, hacer polling de t1, calcular elapsed por resta y comparar contra un umbral, se reutiliza para medir el tiempo de reacción del jugador en cada ronda.

Generación pseudoaleatoria del LED objetivo

Terminada la espera de 3 segundos, el programa debe elegir cuál de los 4 LEDs será el objetivo de la ronda, de forma que la secuencia no sea predecible ni se repita igual en cada partida. El código correspondiente es:

```
lw   x5, CICLOS(x1)      # semilla
xor  x5, x5, x6          # semilla ^ rondas
add  x10, x5, x5          # x10 = 2*semilla
xor  x5, x5, x10          # semilla ^ 2*semilla (mezcla extra)
andi x5, x5, 3           # 0..3
```

Origen de la semilla

La semilla parte del valor instantáneo del contador de ciclos (CICLOS, offset 0x0C), leído en el momento exacto en que termina la espera de 3 segundos. Como el contador avanza en cada flanco de reloj de forma continua e independiente del programa, su valor en ese instante depende del tiempo exacto que llevaba corriendo el sistema — y, en la práctica, del momento en que el jugador encendió la placa o reinició el juego — por lo que no es un valor que el programa pueda predecir de antemano.

Mezcla con el número de ronda

Antes de reducir la semilla a un rango de 2 bits, se combina con x6 (rondas correctas acumuladas) mediante un xor. Esto es importante por una razón concreta: si solo se usara el contador de ciclos como semilla, dos rondas consecutivas que tomen tiempos de reacción muy similares (por ejemplo, el jugador reacciona siempre en un tiempo parecido) producirían semillas muy parecidas, y por lo tanto LEDs objetivo muy correlacionados entre sí. Al mezclar con el número de ronda jugada, se rompe esa correlación: aunque el contador de ciclos avance de forma similar entre rondas, el valor de x6 cambia en cada ronda (0, 1, 2, ..., 9), lo que diversifica la semilla incluso si el jugador reacciona siempre en tiempos parecidos.

Mezcla adicional (difusión de bits)

Después del xor con las rondas, se aplica un paso adicional de mezcla: se calcula 2×semilla (duplicando el valor con una suma, ya que la ALU no soporta shifts) y se hace xor entre la semilla original y ese valor duplicado. Este paso desplaza y combina los bits de la semilla consigo misma, lo que ayuda a que los bits menos significativos del resultado —que es los que finalmente se usan— dependan de una combinación más amplia de los bits originales del contador de ciclos, en lugar de depender solo de sus 2 bits menos significativos directamente.

Reducción a 2 bits y máscara one-hot

El valor final se reduce al rango 0–3 con una máscara andi x5, x5, 3 (quedándose solo con los 2 bits menos significativos). Como la ALU no soporta shifts, este valor de 0–3 no puede convertirse en una máscara one-hot (0001, 0010, 0100, 1000) desplazando bits; en su lugar, el programa usa una cadena de comparaciones y decrementos sucesivos:

```
beqz x5, tgt0
addi x5, x5, -1
beqz x5, tgt1
addi x5, x5, -1
beqz x5, tgt2
j    tgt3
```

Cada bloque tgtN asigna directamente la constante one-hot correspondiente (1, 2, 4 u 8) a x2, que luego se escribe en el registro de LEDs para encender únicamente el LED objetivo.

Cabe señalar que este no es un generador pseudoaleatorio criptográficamente robusto ni con garantías estadísticas formales. Es una mezcla ligera basada en xor y duplicación, pensada para dar variedad perceptible al jugador con el mínimo costo en instrucciones, bajo las restricciones de la ALU del Espino Core no soporta shifts ni multiplicación. Para los fines del juego —evitar que el jugador memorice una secuencia fija— es suficiente, pero no es una fuente de aleatoriedad fuerte.

Limitaciones y decisiones de diseño

Ausencia de shifts en la ALU

El README del repositorio documenta una limitación heredada del core base: las instrucciones de shift se decodifican correctamente pero están deshabilitadas en la ALU para ahorrar LUTs en la FPGA objetivo, y se ejecutan como ADD si se usan. Esta restricción de hardware condiciona directamente el diseño del software del juego: toda operación que normalmente se resolvería con un shift (multiplicar por 2, por 10, o extraer dígitos en base 10 o en base 16 para los displays) debe reemplazarse por sumas repetidas (para multiplicación) o restas repetidas (para división/módulo). Esto se ve reflejado en:

- La generación de la máscara one-hot del LED objetivo (duplicación de la semilla mediante add x10, x5, x5 en lugar de un shift).
- La conversión de ciclos a décimas de segundo (resta repetida de TENTH en lugar de una división).
- La conversión a BCD para los displays (resta repetida de 10, y construcción del valor hexadecimal de dos dígitos con cuatro sumas dobles en vez de un shift de 4 bits).

Esta decisión de diseño prioriza el ahorro de recursos lógicos en una FPGA de gama de entrada (como lo es iCE40 HX1K) por sobre la simplicidad del código assembly, trasladando esa complejidad al software.

RV32E en lugar de RV32I

El Espino Core implementa el perfil RV32E del ISA RISC-V, que define solo 16 registros de propósito general (x0–x15) en lugar de los 32 de RV32I. Esta elección, también heredada del repositorio base, reduce el costo del banco de registros en la FPGA a cambio de menos registros disponibles para el programador. El ensamblador del grupo refleja esta restricción directamente: su tabla de registros (REGS) solo admite x0 a x15, y rechaza con un error explícito cualquier referencia a x16 en adelante.

La RAM unificada del SoC tiene un tamaño fijo de 512 palabras (NumWords en pochoco_ram), por lo que el programa ensamblado (game.hex) no puede exceder esa cantidad de instrucciones. El ensamblador del grupo valida este límite en tiempo de ensamblado y rechaza el programa si lo excede, evitando que un programa demasiado grande se cargue silenciosamente de forma incompleta en memoria.

El Espino Core no implementa un mecanismo de interrupciones, por lo que toda espera de tiempo (los 3 segundos iniciales, la detección de pulsación de botones) se resuelve mediante espera activa: el programa queda en un bucle leyendo repetidamente el periférico correspondiente hasta que se cumple la condición esperada. Esto es simple de implementar y suficiente para los tiempos de este proyecto, pero mantiene al core ejecutando instrucciones sin hacer trabajo útil mientras espera, lo cual sería una limitación relevante en un sistema con múltiples tareas concurrentes, pero en este caso que se implementa un juego simple con tiempos cortos de espera está bien.

Conclusión

La arquitectura descrita en este informe cumple los requisitos de temporización exigidos por

el enunciado del Proyecto 2 apoyándose en una única modificación de hardware: un contador de ciclos de 32 bits agregado al módulo de periféricos, de bajo costo (32 flip-flops y un sumador) y accesible por software en modo de solo lectura. A partir de este contador, y asumiendo un reloj de 25 MHz para la Go Board, se derivaron de forma explícita y verificable las constantes de temporización usadas por el programa (2.500.000 ciclos por décima de segundo, 75.000.000 ciclos para los 3 segundos de espera inicial), y se diseñó un esquema de medición de tiempo por diferencia de lecturas (t1 − t0) que es correcto incluso ante el desborde natural del contador, gracias a las propiedades de la resta en aritmética módulo 2^{32}.

Asimismo, se documentó el mecanismo usado para seleccionar el LED objetivo de cada ronda de forma pseudoaleatoria, combinando el contador de ciclos con el número de rondas jugadas, y se dejaron explícitas las limitaciones de diseño heredadas del core (ausencia de shifts en la ALU, perfil RV32E de 16 registros, tamaño fijo de RAM) que condicionaron varias de las decisiones de implementación tanto en este nivel de arquitectura como en el programa assembly documentado en el Informe 2.

En conjunto, estas decisiones muestran un diseño coherente: se prioriza mantener el hardware simple y de bajo costo en recursos de FPGA, trasladando la complejidad de temporización, aleatoriedad y aritmética al software, lo cual es consistente con el objetivo declarado del Pochoco SoC de ser una arquitectura “entry-level” pensada para entornos con recursos restringidos.